Software Requirements Specification

Use Case Based Capstone Template

Document field	Team entry
Project name	Commerce Bank Personal Finance & Budgeting Assistant
Team name and members	RoyalDev - Cesar Giner, Jacob Melookaran, Joshua Webber, Jake Hall
Sponsor and instructor	Commerce Bank mentors and Shah, Syed Jawad Hussain
Version and status	V0,1 Initial Draft - Waiting Approval
Date	September 29, 2026
Requirements lead	Cesar Giner

How to use this template

This specification contains functional and nonfunctional requirements. Use cases are the primary method for gathering and organizing functional requirements. Nonfunctional requirements define measurable qualities and constraints that apply to the use cases or the product as a whole. Complete both categories before deriving sprint stories, tests, and release decisions. Text inside brackets [ ] tells you what to write. Replace it with your own information before submitting the document.

- Use stable identifiers such as UC-AUTH-01, BR-001, NFR-SEC-001, US-AUTH-03, and TC-AUTH-004.
- Write use cases from the actor's goal and the system's observable response. Avoid screen layout and database design unless they are true constraints.
- Use FR identifiers for separate functional statements and NFR identifiers for measurable quality or constraint statements.
- F• Keep the requirements document as the behavioral source. Link sprint stories and test cases back to a use case flow.
- Record approved changes in the revision history and update traceability during every sprint.

Revision history

Version	Date	Author	Change and approval
0.1	October 2, 2026	RoyalDev Team	Initial actor list, use cases, and catalog

Contents

1 Introduction

2 Stakeholders Actors and Goals

3 Product Context and Constraints

4 Use Case Model

5 Functional Requirements and Detailed Use Cases

6 Nonfunctional Requirements

7 Data and External Interfaces

8 Traceability and Sprint Mapping

9 Review and Change Control

Appendix A Reusable Detailed Use Case Form

1 Introduction

1.1 Purpose and audience

This document provides the formal Software Requirements Specification, detailed use case models, functional requirements, and non-functional specifications for the Commerce Bank Personal Finance & Budgeting Assistant web application. It acts as the definitive technical baseline for software architecture design, sprint breakdown, database modeling, unit and integration testing, and formal stakeholder evaluations.

The target audience includes project sponsors from Commerce Bank, technical course instructors, capstone evaluators, and the RoyalDev engineering team.

1.2 Problem and product vision

Modern consumers, particularly the younger demographic, face significant challenges in building financial literacy, organizing personal budgets, and ability to track their spending.

The Commerce Bank Personal Finance & Budgeting Assistant delivers an intuitive full-stack web application designed to bridge this gap. This platform provides category budget tracking, safe financial transaction simulations, and intelligent AI/LLM contextual insights, empowering users to take control of their financial wellness in a risk-free environment.

1.3 Scope

In scope	Out of scope
User account registration, secure login, authentication persistence, and basic user profile management.	Direct connections to live banking APIs
Simulated transaction logging, payment simulation, and dummy financial card interaction modules.	Real monetary transfers, payment processing networks, or automated clearing house (ACH) operations.
Relational database backend (MySQL) and responsive web interface with React	Real credit card processing or PCI-DSS certification workflows.
Interactive category budget creation, monthly target setting, and visual progress tracking.	
External AI/LLM integration to deliver customized financial advice based on anonymized spending metrics.	

1.4 Definitions and conventions

Term	Definition
Actor	A person, role, device, or external system outside the product boundary that interacts with the product.
Use case	A goal-oriented interaction between an actor and the system.

Term	Definition
Scenario	One path through a use case, including the main success path or an alternate or exception path.
Functional requirement	A capability, rule, validation, calculation, or system response that the product must provide.
Nonfunctional requirement	A measurable quality attribute or constraint on how the product operates or is built.
Must	Mandatory requirement.
Should	Desired requirement that may be deferred with an approved reason.
[Domain term]	[Project-specific definition]

1.5 Assumptions dependencies and open questions

ID	Type	Statement	Owner	Target resolution
A-001	Assumption	Users possess modern web browsers and basic internet connectivity.	RoyalDev Team	[Date/Sprint]
D-001	Dependency	External AI/LLM API service availability	Cesar Giner	[Date/Sprint]
Q-001	Open question	Which specific LLM provider and model version should be targeted for budget insights?	Commerce Bank Mentors/ Instructor	[Date/Sprint]
A-002	Assumption	Database hosting and environment setup will be available early in the project	Jacob Melookaran	[Date/Sprint]
Q-002	Open question	What default budget categories should be pre-populated for new users?	Jake Hall	[Date/Sprint]
[A/D/Q]	[Type]	[Inquiry]	[Owner]	[Date/Sprint]

2 Stakeholders Actors and Goals

Identify people and external systems before naming use cases. An actor represents a role, not a specific teammate or screen.

2.1 Stakeholders

Stakeholder	Interest or decision	How requirements will be validated
Commerce Bank mentors	Sponsor alignment and technical feedback	Midsemester review, questions, and final walkthrough
Instructor	Course expectations and evaluation	Scheduled reviews and submission feedback
RoyalDev Team	Project development	Tasks and deliverables will be validated with team oversight, internal testing and feedback from course instructor

2.2 Actors and user characteristics

Actor ID	Actor or role	Goals	Knowledge permissions and needs
ACT-01	Registered User	Manage monthly category budgets, simulated transactions, track financial goals, and receive budgeting advice.	Standard web user with basic digital experience. Requires authenticated access to personal financial data and an intuitive responsive web interface.
ACT-02	AI/LLM Service	Ingest anonymized spending metrics and return contextual, actionable personal finance recommendations.	External system integration. Must only receive anonymized context without real financial data.
ACT-03	System Administrator	Manage user account statuses, monitor system and API integrations, configure global application parameters, and review system logs.	Elevated administrative privilege and technical proficiency. Requires secure access to admin tools, environment settings, and audit log dashboards.

2.3 Actor goal summary

Actor	Goal	Success outcome	Candidate use case
Registered User	Authenticate and manage personal account details	User account is established and an authenticated session is started	UC-AUTH-01
Registered User	Create and manage category budget targets	Budget targets are stored and displayed in total metrics and visualizers	UC-BUD-01
Registered User	Log simulated financial transactions and payments	Simulated transaction entry is validated, saved, and progress updated	UC-FIN-01
AI/LLM Service	Generate contextual personal finance advice	Actionable financial insights are safely rendered for the user	UC-AI-01
System Administrator	Monitor application operations and manage user accounts	System health metrics and account administrative actions are recorded	UC-ADM-01

3 Product Context and Constraints

3.1 Product environment and system boundary

The system architecture follows a full-stack web application model consisting of three primary layers. The Frontend Presentation Layer is built using React styled with Bootstrap to deliver a fully responsive interface. The Backend Application Server operates as a server responsible for executing core business and application logic, routing, security filters, and LLM prompt generation. Finally, the Database Layer utilizes a relational database PostgreSQL to enforce strict data integrity through normalized schemas and primary/foreign key constraints.

3.2 Sponsor brief baseline

• Build a full-stack web application selected by the team; do not use a CMS such as WordPress.
• Include a meaningful financial component such as budget tracking, simulated payments, or investments.
• Support user registration and login; store passwords as secure hashes, never as plain text.
• Use a database, a front-end framework or library, and one CSS framework.
• Provide Login/Register, Home/Dashboard, User Profile/Settings, and at least two additional pages.
• Use only fake or dummy card and financial data when simulating transactions.
• Include unit testing; automated tests are strongly encouraged.

3.3 Chosen stretch goals

Stretch goal	Reason for selection	Use cases or requirements affected	Evidence
Data Visualization	Provides users with intuitive, interactive visual charts to track spending trends over time and gain deeper financial insights.	UC-BUD-01	Interactive dashboard demo showing monthly category breakdown and trend analysis charts.
Persistent User Sessions	Improves user experience and security by maintaining authenticated sessions	UC-AUTH-02/03	Automated test suite verifying session persistence after page refreshes

3.4 Business rules

Rule ID	Rule	Source	Related use cases
BR-001	Development, testing, and demonstrations must use fabricated financial and card data.	Sponsor brief	UC-FIN-01, UC-AI-01
BR-002	User passwords must be securely hashed using salted algorithms prior to persistence.	Security policy	UC-AUTH-01
BR-003	The solution must be built as a custom full-stack web application; off-the-shelf website builders or CMS platforms are not permitted.	Course guidelines	All use cases
BR-004	Prompts sent to external AI services must strip out all private information and transmit only relative, anonymized spending and category metrics.	Privacy Policy	UC-AI-01
BR-005	[Rule]	[source]	[Use Case]

4 Use Case Model

4.1 Use case diagram

Insert a readable diagram that shows actors outside the system boundary and goal-oriented use cases inside it. Use include or extend only when the relationship improves understanding.

External Actors (Left)	System Boundary: Application System	External Actors (Right)
ACT-01: Registered User *(Primary User)*	• UC-AUTH-01: Register an account • UC-AUTH-02: Sign in • UC-AUTH-03: Persistent User Sessions • UC-BUD-01: Create a monthly budget • UC-FIN-01: Log simulated transaction • UC-AI-01: Generate AI budget insights • UC-PROF-01: Display Profile Info and Settings	ACT-02: AI/LLM Service *(External System)*
System Administrator *(Admin)*	• UC-ADM-01: Monitor operations & accounts	-

NOTE: UseCase Document in Github repo for a more detailed information for each Usecase

4.2 Use case catalog

ID	Use case name	Primary actor	Priority	Status	Target	Owner
UC-AUTH-01	Register an account	Registered user	Must	Draft	Sprint 1	RoyalDev Team
UC-AUTH-02	Sign in	Registered user	Must	Draft	Sprint 1	RoyalDev Team
UC-AUTH-03	Persistent User Sessions	Registered user	Must	Draft	Sprint 2	RoyalDev Team
UC-BUD-01	Create a monthly budget	Registered user	Must	Draft	Sprint 2	RoyalDev Team
UC-FIN-01	Log simulated transaction	Registered user	Must	Draft	Sprint 3	RoyalDev Team
UC-AI-01	Generate AI budget insights	AI/LLM Service	Must	Draft	Sprint 3	RoyalDev Team
UC-ADM-01	Monitor operations & accounts	System administrator	Should	Draft	Sprint 3	RoyalDev Team
UC-PROF-01	Display Profile Info and Settings	Registered user	Must	Draft	Sprint 2	RoyalDev Team

4.3 Use case relationships and scope notes(REVIEW)

Relationship or boundary	Explanation and reason
[UC-A includes UC-B]	[Explain reusable required behavior; remove this row if include is not needed]
[UC-X extends UC-Y]	[Explain optional or conditional behavior; remove this row if extend is not needed]
Out-of-scope actor goal	[Record a goal discussed with stakeholders that will not be implemented and why]

5 Functional Requirements and Detailed Use Cases

Use cases are the primary source of functional requirements. Copy the reusable form in Appendix A for every high-priority use case. Complete the main success scenario and realistic alternate and exception flows before

the related story enters a sprint. Add a separate FR statement when behavior applies across use cases or needs more precision than a flow step provides.

5.1 Functional requirements catalog

Use this catalog for required behavior that applies across use cases or needs a precise standalone statement. Sponsor-brief capabilities such as required pages belong here, not in the nonfunctional section. Every functional requirement must link to at least one use case, business rule, or sponsor constraint and must be testable.

FR ID	Functional requirement	Source or related use cases(UseCase ID)	Priority	Verification
FR-AUTH-01	The system must allow a prospective user to register for an account	UC-AUTH-01	Must	TC-AUTH-01
FR-AUTH-02	The system must allow a registered user to sign into the application on the login page	UC-AUTH-01, UC-AUTH-02	Must	TC-AUTH-02
FR-AUTH-03	Webpages should retain cookies on the user in order to keep the session persistent	UC-AUTH-03	Must	TC-AUTH-03
FR-BUD-001	The system shall provide the user with a template for a monthly budget that afford them savings each month	UC-BUD-001 UC-FIN-01	Could	TC-BUD-01
FR-FIN-01	The system will be able to keep track of individual transactions for users for a certain time period (60 days default)	UC-FIN-01	Must	TC-FIN-01
FR-AI-01	Given a customer report of transactions and deposits, an AI/LLM will be able to generate a budget projection for the next month	UC-AI-01 UC-FIN-01	Should	TC-AI-01
FR-ADM-01	The system should be able to be monitored by an administrator who can see all accounts and activity	UC-ADM-01	Must	TC-ADM-01
FR-PROF-01	When a user is logged in and selects their profile information and settings page, it should display all the relevant information	UC-PROF-01	Must	TC-PROF-01

6 Nonfunctional Requirements

State measurable quality attributes and constraints that apply across the product or to named use cases. Every NFR must include operating conditions, a target or threshold, and a verification method. Do not write vague statements such as the system must be fast or secure.

ID	Category	Requirement and measure	Verification	Priority
NFR-SEC-001	Security	Passwords must be stored only as salted hashes produced by the selected authentication library; plain-text passwords must not appear in storage or logs.	Integration inspection and log review	Must
NFR-PRIV-001	Privacy	The application and all demonstrations must use fabricated financial and card data and must not request real banking credentials.	Repository database and demo review	Must
NFR-USE-001	Usability	The majority of first-time users must be able to create	[Usability test]	[Priority]
NFR-PERF-001	Performance	[Operation, dataset or load, response target, and percentile]	[Measured test]	[Priority]
NFR-ACC-001	Accessibility	Text within the pages of the application must be adjustable to alleviate any visual impairments.	[Audit or manual test]	[Priority]

ID	Category	Requirement and measure	Verification	Priority
NFR-REL-001	Reliability	[Failure behavior, recovery, or availability target]	[Test or observation]	[Priority]
NFR-MNT-001	Maintainability	[Test, review, modularity, or configuration expectation]	[Review or pipeline evidence]	[Priority]

NFR ID	Quality requirement and measure	Related use cases	Verification
NFR-UI-001	For the selected representative workflows, all interactive controls must be reachable and operable by keyboard.	[UC IDs]	Keyboard walkthrough
NFR-UI-002	Labels, validation messages, and navigation must be understandable to representative users completing the selected tasks.	[UC IDs]	Usability test with stated success criterion
NFR-UI-___	[Measurable interface quality or accessibility constraint]	[UC IDs]	[Method and threshold]

7 Data and External Interfaces

7.1 Data requirements

Data ID	Entity or data	Important fields and validation	Ownership privacy and retention	Related use cases
DATA-USER	User account	ID, fullname, username, email, passwordHash, isAdmin, AccountCreationTime		[UC IDs]
DATA-AI	AI			
DATA-Transactions				
DATA-Tokens	[Entity]	[Fields, types, ranges, uniqueness]	[Rules and prohibited data]	[UC IDs]

7.2 External interfaces

Interface ID	System or service	Purpose and contract	Authentication data and failure behavior	Owner
INT-001	[API, reporting tool, email service, or browser]	[Inputs, outputs, protocol, version]	[Trust, timeout, retry, unavailable behavior]	[Name]
INT-002				

8 Traceability and Sprint Mapping

8.1 Sprint backlog mapping

Jira Framework for Project Sprint Mapping:
https://umkc-team-hlh86r9q.atlassian.net/jira/software/projects/CP/list?jql=project%20%3D%20CP%20ORDER%20BY%20cf%5B10019%5D%20ASC

8.2 Prioritization

Explain how Must, Should, and Could priorities affect the MVP and sprint order. Mandatory sponsor requirements must be completed before optional stretch work. Record architecture or risk-driven sequencing when it differs from business priority.(Document as we move forward)

9 Review and Change Control

9.1 Review checklist

- [ ] Every actor has at least one goal and every in-scope goal maps to a use case.
- [ ] Each detailed use case has a trigger, preconditions, main success scenario, alternate or exception flows, and postconditions.
- [ ] The use case describes behavior and outcomes without unnecessary screen or database design.
- [ ] Functional requirements describe testable behavior and use stable FR identifiers when standalone statements are needed.
- [ ] Nonfunctional security, privacy, usability, performance, accessibility, and reliability requirements include measurable targets and verification methods.
- [ ] Use cases, FRs, and NFRs link to sprint stories and test cases by stable identifier.
- [ ] All examples, fixtures, screenshots, and demonstrations use fabricated financial data.

9.2 Change requests and decisions

Change ID	Requested change and reason	Affected IDs	Decision and approver	Date sprint
CHG-001	[Request and business or technical reason]	[UC BR NFR DATA IDs]	[Approved deferred rejected and by whom]	[Date / sprint]

Appendix A Reusable Detailed Use Case Form

Copy this entire appendix section for each use case. Replace every bracketed prompt and delete rows that do not apply only after the team confirms they are unnecessary.

Field	Team entry
Use case ID and name	[UC-AREA-## Goal-oriented verb phrase]
Goal	[Actor value]
Primary actor	[Actor ID and role]
Supporting actors	[Roles or external systems]
Trigger	[Starting event]
Preconditions	[Required prior state]
Success postconditions	[State after success]
Minimum guarantee	[State preserved after failure]
Priority status target owner	[Entries]

Main success scenario

Step	Actor or system	Action or response
1	[Actor/System]	[Action or observable response]
2	[Actor/System]	[Action or observable response]
3	[Actor/System]	[Action or observable response]
4	[Actor/System]	[Action or observable response]
5	[Actor/System]	[Action or observable response]
6	[Actor/System]	[Action or observable response]

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
__a	__	[Condition]	[Behavior and return point]
__b	__	[Condition]	[Behavior and return point]
__c	__	[Condition]	[Behavior and return point]

Rules data quality and evidence

Item	Linked IDs or details
Business rules	[BR IDs]

Item	Linked IDs or details
Data inputs and outputs	[Data IDs and validation]
Functional requirements	[FR IDs]
Nonfunctional requirements	[NFR IDs]
Acceptance and test evidence	[TC IDs and acceptance examples]
Open questions and assumptions	[Q and A IDs]
Backlog mapping	[Story or task IDs with scenario slices]

Use case review record

Review field	Entry
Owner and last updated	[Name and date]
Reviewers	[Requirements reviewer, developer, tester, sponsor or instructor when applicable]
Status and target	[Proposed / Draft / Approved; target sprint or release]
Decision or follow-up	[Approved, changes requested, deferred question, and linked issue IDs]

[ ] The primary actor and goal are clear.

[ ] Success, alternate, and failure behavior are testable.

[ ] Business rules, functional requirements, nonfunctional requirements, sprint work, and tests are linked by ID.

[ ] The use case is ready for the target sprint only after review issues are resolved.